From Generating SQL to Trusting SQL: A Systematic Review of Reliability Mechanisms in LLM-based Text-to-SQL Systems

Ke Lu
UNI: kl3753

Abstract

In recent years, large language models (LLMs) have developed rapidly, transforming many traditional industrial models, including SQL. LLMs have advanced the development of Text-to-SQL technology, enabling non-technical personnel to use natural language instead of SQL query statements. The Text-to-SQL technology currently implemented has achieved high execution accuracy on complex datasets such as Spider. However, after reading many representative papers, I found that although the current research has continuously improved in performance metrics, it lacks discussion on the reliability of the generated results. To avoid confining the discussion on reliability to the level of bigger models and higher scores, this paper, based on a comparative analysis of multiple LLM-based Text-to-SQL and related database interface studies, examines the literature from the perspectives of Schema Awareness, Guardrails & Verification, and Knowledge & Reasoning. It explores the commonalities and differences among various studies, as well as my insights and understanding of Text-to-SQL after the research. I prefer to view these three mechanisms as solutions to problems rather than mere model comparisons. I will focus on the specific changes of Text-to-SQL under different models, discussing what types of errors each mechanism can eliminate, what new risks they may introduce, and how they can be combined in real-world scenarios to enable users not only to get it running but also to trust the results. This paper aims to re-examine the field from the perspective of shifting from generating correctly to being trustworthy.

1. Introduction

The objective of Text-to-SQL is to lower the threshold for database usage, enabling non-technical personnel to access structured data through natural language. Currently, related research has achieved significant performance improvements on datasets such as Spider. Many papers take Execution Accuracy as the primary evaluation metric, and they all measure the success of the model through Execution Accuracy.

However, I believe we need to question the sufficiency of this metric. Execution Accuracy only ensures that the generated statements can be executed, but this does not mean that the query is necessarily valid. In a real database environment, a statement may produce misleading results even if it executes successfully, due to missing conditions, incorrect aggregations, or unreasonable join path selections. This phenomenon of "running correctly but semantically incorrect" is often difficult to expose in benchmark datasets. Even for non-technical personnel, it is hard for them to determine whether the final result is truly correct, and they can only blindly trust the model.

Execution Accuracy depends on whether the output structure set is consistent, but it is not strongly correlated with semantic errors. In actual experiments, even if the execution result set of SQL is consistent with the labeled answer, it is possible that LLMs have misunderstood the semantics but coincidentally output the correct answer. Common situations include: missing filter conditions but not affecting the current data distribution, inconsistent aggregation criteria but having the same value in the sample, or an unreasonable join path but not introducing additional rows. The RBDQ paper pointed out this risk when defining EX: queries with different semantics may obtain the same result set, thereby causing "false negatives/false sense of security" in the evaluation. [1]

Non-technical users find it difficult to determine whether the final result is correct. Even professionals sometimes struggle to ascertain if the final values are derived from the intended table. On this basis, non-technical personnel cannot audit the correctness by reading SQL, nor do they have the ability to check the schema or reproduce the join logic. Therefore, this technology requires extremely high accuracy and a reliable mechanism that enables users to have verifiable evidence to understand why the system selected these tables, why this condition was added, where the result might be wrong, and how users can verify or correct it at the lowest cost.

Therefore, I am more inclined to organize the relevant studies from the perspective of reliability mechanisms and analyze how they jointly or separately affect the credibility of the system. Our main goal is to make it accessible to non-technical users. However, being functional does not directly translate into a reliable indicator. The evaluation indicators need to be more closely aligned with the error patterns and interpretability requirements in real usage scenarios.

What the Reader Is Expected to Learn

By reading this article, readers will understand that:

- Classify the failures of Text-to-SQL according to structural errors, constraint deficiencies, and semantic deviations;
- The current LLM-based Text-to-SQL methods can be analyzed from three reliability mechanisms;
- Different papers focus on different aspects such as structural constraints, verification mechanisms, and semantic reasoning;

- The existing research still has deficiencies in integration across mechanisms; Performance indicators cannot fully reflect the reliability of real-world applications.

2. Background

The current LLM-based Text-to-SQL methods usually are based on pre-trained language models and achieve SQL generation through prompt design or fine-tuning. In current research, some tend to utilize the language understanding ability of large language models and directly concatenate the database schema and the question as input to the model, while others prefer to incorporate structural constraints or verification modules in the generation process.

In the recent two years of research, a new trend has emerged, which is the transformation of the process from single-time generation of SQL to multi-stage proxy. For instance, in some systems, the model first performs schema linking or query classification, then generates SQL, and then self-corrects or uses external tools for verification. ToolSQL clearly states that many multi-stage methods rely on the limited feedback provided by the database management system for error correction, but there is insufficient exploration of "stronger verification and correction", which also reminds us that reliability is not just about writing a good prompt, but requires incorporating verification as a first-class citizen in the architectural design. [10]

In addition, I found that many studies have begun to emphasize disturbances other than benchmarks and real noise. ToolSQL built SpiderMismatch to simulate common condition mismatch problems in reality and demonstrated that when the problem statement is perturbed or the condition values do not match the real units in the database, the performance of many methods will significantly decline. Therefore, we cannot only conduct experiments on simple and limited datasets; we must use more realistic and closer-to-real-life datasets for experiments.

After reading two review papers, "Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL" [5] and "Text-to-SQL: A Methodical Review of Challenges and Models" [7], I discovered that the main challenges to the reliability of Text-to-SQL can be categorized as problems related to structure understanding, semantic understanding, and generation stability. Therefore, I will classify the relevant papers from these three perspectives, and then study the commonalities and differences of each category.

3. Schema Awareness

In these studies, understanding the structure was the issue that was emphasized the earliest. In the earlier methods based on LLM, the complete database schema was often directly used as the prompt input to the model. This approach expected the model to automatically identify the relevant tables and columns through language patterns. However, in datasets like Spider that contain complex multi-table relationships, injecting the complete schema often introduces a large amount of irrelevant information to the current problem, thereby increasing the risk of incorrect column selection. Multiple papers have reported that when the schema size is large, the model is more prone to incorrect matching.

In contrast, PSM-SQL proposes a multi-stage schema linking mechanism [11], which reduces structural noise by gradually filtering out tables and columns. The core idea is to first filter out relevant tables based on the problem semantics, and then further match specific columns. This staged constraint approach demonstrates greater stability in complex query scenarios.

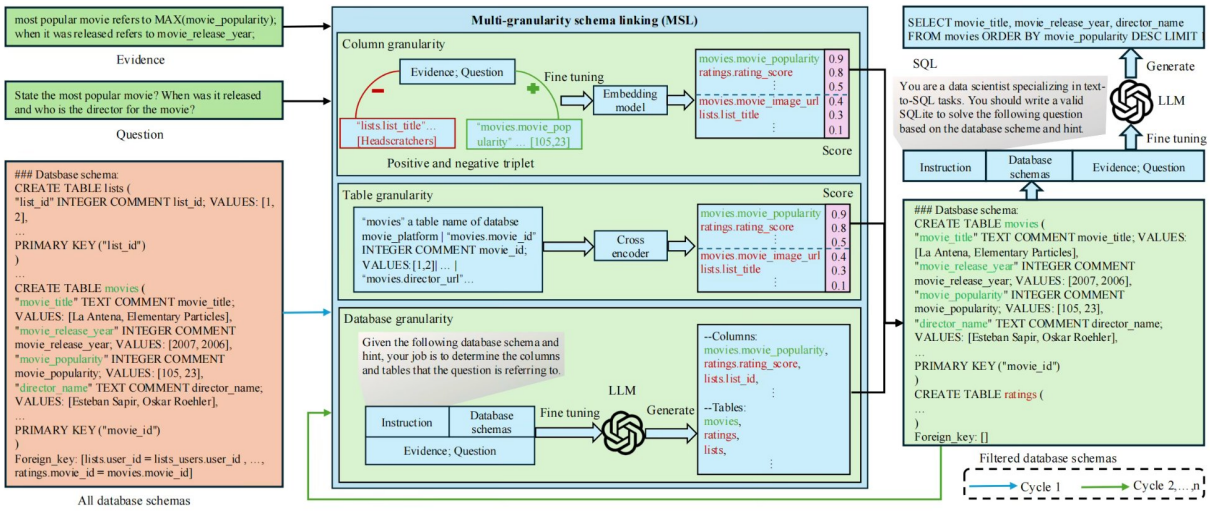

Figure 1: The multi-stage schema linking framework diagram of PSM-SQL [11].

As shown in Figure 1, PSM-SQL reduces the candidate space through structural filtering, thereby enhancing the stability of the generation stage. However, such methods also have limitations. If there is a deviation in the semantic matching stage, the system may lose key structural information before the generation process. It does not simply truncate the schema; instead, it treats schema linking as a chain-like task that can be gradually degraded in difficulty: at the column level, use triplet loss to fine-tune the embeddings, making the correct columns closer to the problem semantics and the incorrect columns being pulled further apart; at the table level, combine the classifier and similarity scores to model the inter-table interactions; at the database level, let the LLM learn "which schemas are relevant to the generation". At the same time, it also introduces the chain loop strategy, allowing for a certain degree of redundancy but continuously reducing irrelevant schemas, thereby maintaining recall while improving precision. [11]

This structural choice transforms the previously unknown and uncontrollable operations into controllable ones, reducing the possibility that LLMs will make semantic errors, fail to detect them, and proceed to the next step.

Furthermore, some studies attempt to enhance the schema representation capability through graph structure encoding, embedding the table column relationships into the internal structure representation of the model. These methods are theoretically more rigorous, but they have higher training complexity and require higher data scales. At the same time, if there is a deviation in the early stage of schema linking, even if the subsequent generation and reasoning are very strong, the final result cannot be correct. We must make the schema linking step transparent and public, so that we can operate to reduce semantic errors. For example, we can output the confidence ranking of candidate table columns, give the reasons for filtering out the schema, or allow rollback and re-search in key steps, making the structure selection a verifiable and correct process, rather than a one-time decision.

As summarized, Schema Awareness addresses the issue of whether the structural alignment is correct. However, having only this guarantee is not sufficient; we also need other verification mechanisms.

4. Guardrails & Verification

After Schema Awareness, some studies began to focus on how to introduce verification mechanisms after the generation stage. The most common method is execution-guided decoding, where the SQL is executed immediately after generation. If the execution fails, it is re-generated. With this mechanism, the number of syntax errors has significantly decreased, and the Execution Accuracy has also been improved. However, this approach only filters out unexecutable statements and is powerless against executable but logically incorrect statements.

However, although execution-guided can significantly reduce direct errors, it is still essentially verifying whether the program can run, rather than verifying semantic errors. The code examples previously mentioned, such as missing filtering conditions but not affecting the current data distribution, inconsistent aggregation standards but the values being the same on the sample, or an unreasonable join path but without introducing additional rows, can all run but result in deviations. These are errors that execution-guided cannot detect. Execution-guided is more like a grammar filter rather than a semantic safety net.

ToolSQL introduces external tools for rule checking after its generation [10], such as checking whether fields exist and whether joins are reasonable. This approach is more feasible in terms of engineering compared to execution-guided methods.

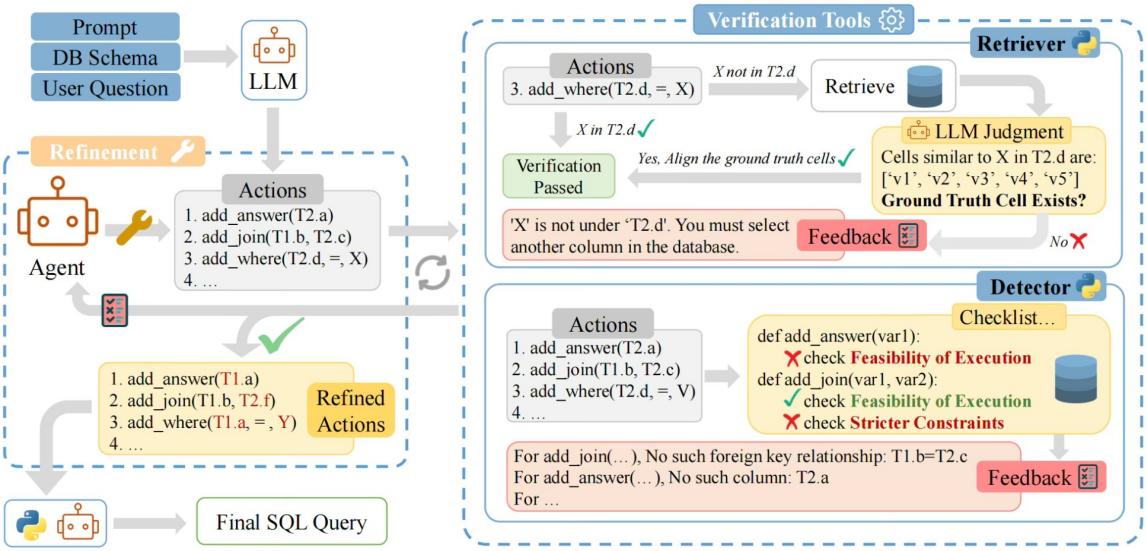

Figure 2: The generation-verification flowchart of ToolSQL [10].

As shown in Figure 2, ToolSQL adds a tool layer between generation and execution, attempting to enhance reliability through rule detection. However, this approach still relies on manual rule design and is difficult to cover complex business semantics. ToolSQL divides the validation into two types of tool signals: one focuses on the matching between condition values and database units (Database Retriever), and the other focuses on more stringent constraints and execution exceptions (Error Detector). It uses an LLM-based agent to generate and continuously refine actions. When the tool detects issues such as the non-existence of a join, the non-existence of a column, or the condition value not being in the target column, it will return targeted feedback to drive the agent to iteratively correct. [10]

Another type of research, such as STaR-SQL [3], introduces a self-refinement mechanism, allowing the model to conduct self-evaluation after generation and regenerate itself. These methods leverage the reflective ability of LLMs and achieve certain improvements in complex queries.

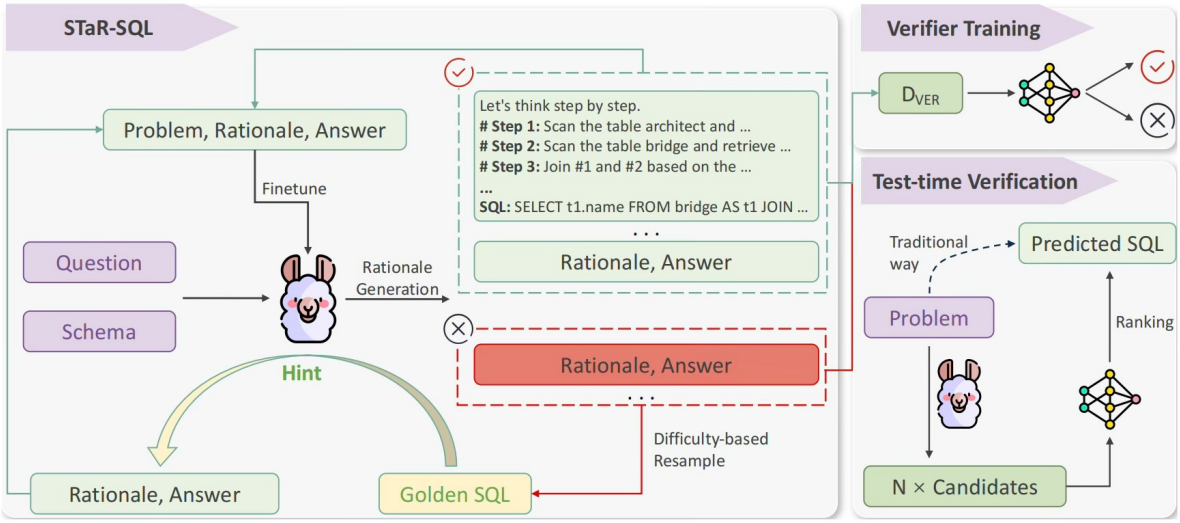

Figure 3: Self-training flowchart of STaR-SQL [3].

However, all these studies only focus on the correctness of form, and still lack systematic methods for the consistency of deep semantics. Guardrails can be more accurately understood as a necessary but insufficient safety layer: It can significantly reduce avoidable low-level errors, but it cannot replace the modeling and interpretation of business semantics.

5. Knowledge & Reasoning

After studying Guardrails & Verification, I discovered that many Text-to-SQL errors are not structural or grammatical issues, but rather stem from misunderstandings of the problem's semantics. For instance, "top customer" could refer to the customer with the highest income, the one with the most orders, or the one with the highest recent spending. Relying solely on the schema cannot determine its meaning.

The issues in this part have nothing to do with SQL. Non-computer technicians in various industries have their own industry-specific definitions of data in their respective fields, such as: "active users", "productivity", "defect rate", and even some indicators that rely on professional knowledge from other fields or industry default indicators. This knowledge is something that our computer programmers cannot think of or train for in advance. When encountering related problems, the model can only guess the user's intentions within the "field combination space", which is prone to generating "seemingly reasonable" illusionary queries. Therefore, the goal of the Knowledge & Reasoning mechanism is not only to make the model think a few more steps, but also to enable it to explicitly express the implicit definitions in natural language and trace back to the verifiable knowledge sources.

RBDQ presents a very typical "trust-oriented" engineering approach: It acknowledges that LLMs may overlook conditions or deviate from the planned steps during the chain-of-thought planning process, thereby resulting in "executable but unreliable" SQL. Therefore, it uses the "indicator

template + predefined calculation operators" method to map natural language queries to controllable metric SQL templates, and tries to place the calculation logic in the code implementation rather than having LLM generate executable code. The direct benefit of this approach is that even if there are errors in the planned steps, business users can more easily locate the problem along the steps, determine whether the result is usable, and thereby improve traceability and auditability. [1]

The true function of the reasoning mechanism lies in explicitly expressing the uncertain and ambiguous assumptions and questions before generating the SQL query, rather than allowing them to remain hidden in the data and proceed to the next query, ultimately arriving at the final conclusion. Chain-of-Thought prompting not only improves the accuracy of the final generated results, but has also been proven to make some decision processes visible [9]. By explicitly generating intermediate reasoning steps, the model can organize logical relationships more clearly. Researchers or users can more easily check whether the model has deviated from the user's intention when understanding the user's intent, without having to wait until seeing the final SQL to identify the problem. This to some extent improves the interpretability of the system. However, such methods also have limitations. The longer the reasoning chain, the higher the generation cost, and the risk of error propagation will also increase. Even if the reasoning process is clearly visible, users can judge whether the reasoning is correct, but if the initial basis of the reasoning is incorrect, the final reasoning result may still be incorrect. Therefore, in system design, the reasoning mechanism is more suitable to be used in combination with the verification mechanism rather than being regarded as a substitute for verification.

STaR-SQL [3] further improves the model's reasoning ability by re-training it with self-generated reasoning samples, making the reasoning more stable. However, this method requires a relatively high amount of training resources. Specifically, the core idea of STaR-SQL is to utilize the reasoning process (rationale) generated by the language model itself as new training data, thereby continuously improving the model's reasoning ability. During the training phase, the model first generates a series of candidate answers containing reasoning steps and SQL queries for the problem, and then selects the correct reasoning-answer pairs through execution results or verification modules. These verified correct reasoning samples are subsequently re-used for model fine-tuning, enabling the model to gradually learn more stable and consistent reasoning patterns. Through this self-training loop, the model can continuously improve its reasoning ability without the need for a large amount of manually annotated reasoning data, thereby enhancing its performance in complex query tasks. However, this method also faces some new problems. Generating and filtering reasoning samples require a significant amount of computing resources because the model needs to repeatedly generate candidate reasoning paths and execute corresponding SQL for verification. Additionally, if the reasoning process generated by the initial model is already biased, this bias will be infinitely amplified during the training process. Therefore, STaR-SQL typically requires the introduction of additional verification mechanisms or filtering strategies to ensure that the data used for re-training has a higher quality.

The Knowledge & Reasoning mechanism and Schema Awareness do not exist independently. Improving reasoning ability cannot compensate for structural alignment errors, and structural constraints cannot replace business semantic understanding. The intermediate steps generated by reasoning can, in turn, provide stronger semantic anchors for schema linking, and also offer more clear inspection targets for verification tools. Most current papers still optimize different aspects separately and lack a systematic integration framework.

6. Intersections and System-Level Integration

In conclusion, the reliability of Text-to-SQL is not determined by a single mechanism; it requires the simultaneous application of three mechanisms. Schema Awareness is responsible for structure selection, Guardrails provide generation constraints, and Knowledge & Reasoning are responsible for semantic understanding. Most studies only utilize one or two of these mechanisms, and very few studies have evaluated the effect of using all three mechanisms in synergy.

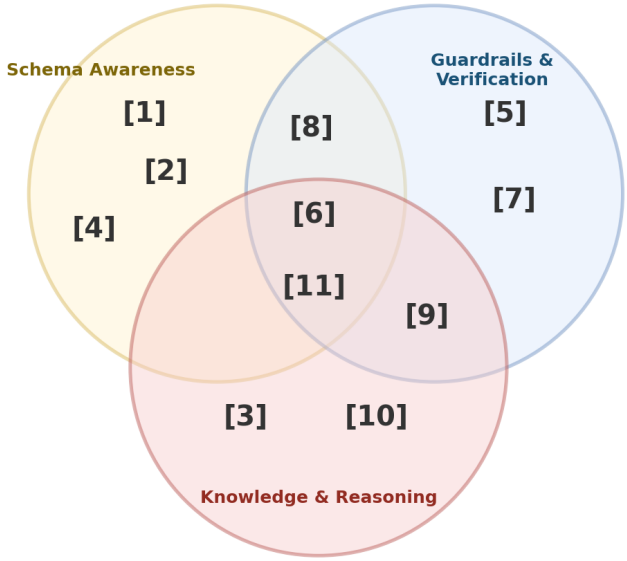

Figure 4: Venn diagram of the three reliability mechanisms and the placement of reviewed papers.

By integrating the three types of mechanisms into the same system pipeline, a more realistic and usable Text-to-SQL architecture can be achieved. The system first uses the structure-aware method to narrow down the candidate schemas and outputs an interpretable candidate ranking; then the model generates "intent and description" based on the candidate schemas, and then converts the description into a SQL draft; finally, the tool layer performs consistency verification and correction, focusing on intercepting frequent errors such as mismatched condition values, non-existent columns/tables, and unreachable joins, and then proceeds to execution and result display. In this way, during the process of system generation, users can detect semantic understanding errors of the system, modify the explanation of the purpose in advance, and provide more evidence. This transparent and operable Text-to-SQL generation process is therefore more reliable.

7. Conclusion

This paper organizes the literature from the perspective of reliability and analyzes the commonalities and differences among these papers.

By reading multiple LLM-based Text-to-SQL papers, I found that performance improvement is not equivalent to reliability improvement. I believe that merely using the EM/EX data indicators cannot prove the reliability of the system and cannot detect whether there are semantic errors. A more reasonable evaluation method should include robustness after semantic perturbation or value perturbation of the input questions [10] (similar to the SpiderMismatch approach in ToolSQL); for the fallback strategy when business definitions are missing [1], such as suggesting the lack of indicator definitions and requesting user confirmation; and for the assessment of the availability of explainable evidence [3], to detect whether users can quickly determine the correctness using the explanations provided by the system. Only in this way can the generation process be made transparent, allowing users to promptly realize semantic errors instead of the system generating incorrect answers with errors.

Structure alignment, generation verification, and semantic reasoning are all indispensable, and current research still lacks in the aspect of cross-mechanism integration. When the system is targeted at non-technical users, the true criteria for success are: whether the system can provide an auditable path, whether it can proactively expose risks in uncertain situations, and whether it can correctly downgrade in the absence of business definitions.

The main gap of the current research lies in not taking reliability as the primary research objective but instead blindly attempting to enhance the performance of the model. A more promising direction for future investment might be to integrate schema linking, reasoning, and tool-based verification into a learnable, measurable, and interpretable framework, and test it on more realistic data and more realistic error distributions.

What I Learned

Before reading these papers, I always thought that Text-to-SQL was a very straightforward topic. With the prevalence of LLMs nowadays and their powerful capabilities, it should be able to handle simple tasks like SQL queries easily. During my learning process, I felt that SQL code was much simpler compared to other types of code. However, after reading these papers, I realized that my assumption was too simplistic. The content of the SQL database schema is time-consuming, which poses a significant challenge for LLMs. Additionally, in our work, the accuracy of SQL queries must be extremely high, and it is also difficult for us to identify incorrect query answers. Suddenly, I realized that if I queried the price and Text-to-SQL gave me an incorrect price, it would be very difficult for me to identify it. These issues made me realize that this is a very urgent and necessary topic. At the same time, I also learned about three very ingenious verification mechanisms: structure alignment, generation verification, and semantic reasoning. All of these have greatly benefited me. I realized that Text-to-SQL is a system engineering problem, not just a language generation problem. The differences between the papers often lie in the different focus points rather than simple model size differences.

At the same time, I discovered that the real problem with Text-to-SQL does not lie in the difficulty of SQL, but rather in whether non-technical users can accurately express their requirements and whether the system can accurately grasp the users' meanings. In fact, the fundamental goal of these current research models is to find ways to enable non-technical users to collaborate with the system

to jointly achieve the final correct results. Instead of merely improving the system's comprehension ability, it would be better to allow users to easily join in and jointly determine the reliability of the results. This reading process has changed my understanding of "performance being success".

References

[1] Fenglin Bi, Dongdong Cao, Zhiyu Wang, Yang Chen, Fangliang Zhao, Tao Hu, Zhi Li, Yanbin Zhang, and Wei Wang. RBDQ: A reliable LLM-based text-to-SQL system for business data queries. In *Companion Proceedings of the ACM on Web Conference 2025*, WWW '25, pages 95–103, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400713316. doi: 10.1145/3701716.3715257. URL https://doi.org/10.1145/3701716.3715257.

[2] Dawei Gao et al. Text-to-SQL empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023. doi: 10.48550/arXiv.2308.15363. URL https://doi.org/10.48550/arXiv.2308.15363.

[3] Mingqian He, Yongliang Shen, Wenqi Zhang, Qiuying Peng, Jun Wang, and Weiming Lu. STaR-SQL: Self-taught reasoner for text-to-SQL. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24365–24375, Vienna, Austria, 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1187. URL https://aclanthology.org/2025.acl-long.1187/.

[4] Zijin Hong, Zheng Yuan, Hao Chen, Qinggang Zhang, Feiran Huang, and Xiao Huang. Knowledge-to-SQL: Enhancing SQL generation with data expert LLM. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10997–11008, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.653. URL https://aclanthology.org/2024.findings-acl.653/.

[5] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. Next-generation database interfaces: A survey of LLM-based text-to-SQL. *IEEE Transactions on Knowledge and Data Engineering*, 37(12):7328–7345, 2025. doi: 10.1109/TKDE.2025.3609486. URL https://ieeexplore.ieee.org/document/11160657.

[6] Mohammed Jabir, Kamal Kanakarajan, and Malaikannan Sankarasubbu. Saama technologies at EHRSQL 2024: SQL generation through classification answer selector by LLM. In *Proceedings of the 6th Clinical Natural Language Processing Workshop*, pages 655–671, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.clinicalnlp-1.63. URL https://aclanthology.org/2024.clinicalnlp-1.63/.

[7] A. Kanburoğlu and F. B. Tek. Text-to-SQL: A methodical review of challenges and models. *Turkish Journal of Electrical Engineering and Computer Sciences*, 32(3):403–419, 2024. doi: 10.55730/1300-0632.4077. URL https://doi.org/10.55730/1300-0632.4077.

[8] Wenxin Mao, Ruiqi Wang, Jiyu Guo, Jichuan Zeng, Cuiyun Gao, Peiyi Han, and Chuanyi Liu. Enhancing text-to-SQL parsing through question rewriting and execution-guided refinement. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2009–2024, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.120. URL https://aclanthology.org/2024.findings-acl.120/.

[9] Ahmed Yasser Mohamed, Mohamed Hisham Zain-elabdeen, Salwa Ahmed Sayed, Samaa Sabry Abdelwahab, Ziad Ezzat Mohamed, Zyad Mahmoud Ali, Yomna A. Kawashti, and Hanan Hindy. SQLify Me: An automated text-to-SQL query generator. In *2024 34th International Conference on Computer Theory and Applications (ICCTA)*, pages 297–302, 2024. doi: 10.1109/ICCTA64612.2024.10974783. URL https://ieeexplore.ieee.org/document/10974783.

[10] Zhongyuan Wang, Richong Zhang, Zhijie Nie, and Jaein Kim. ToolSQL: A tool-assisted agent for SQL verification and refinement. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD '25, pages 3102–3110, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400714542. doi: 10.1145/3711896.3737159. URL https://doi.org/10.1145/3711896.3737159.

[11] Zhuopan Yang et al. PSM-SQL: Progressive schema learning with multi-granularity semantics for text-to-SQL. *arXiv preprint arXiv:2502.05237*, 2025. doi: 10.48550/arXiv.2502.05237. URL https://doi.org/10.48550/arXiv.2502.05237.